Cybersecurity Practical: Mini Projects / Practical Output

# Mini Project 3: AI-Assisted Cybersecurity Threat Model

| | |
|---|---|
| **Student name** | Neralla Sairam Suraj |
| **Roll number / Student Code** | STU-B33F7C |
| **Course / Group** | Cyber Security / G17 |
| **Platform** | SURE ProEd |
| **Host OS** | Windows 10 |
| **Date** | 22/10/2026 |

## 1. Aim

To analyse cybersecurity threats in the **PhishGuard AI** multi-tier phishing protection and threat modeling system using the STRIDE methodology and AI-assisted recommendations. The report uses the supplied runtime screenshots, source code implementations, and automated test outputs as evidence and does not treat AI recommendations as proof of implemented controls.

## 2. Objectives

- Review the multi-tier PhishGuard AI architecture encompassing a Google Chrome Extension (Manifest V3), Content Script interceptor, Python Flask web service, and SQLite database.
- Analyse four critical threats affecting the client browser navigation, DOM execution context, Web Service, and audit database components.
- Classify each threat systematically using the STRIDE threat-modeling methodology.
- Use AI to explain attack scenarios, impacts, mitigations, limitations and verification evidence.
- Compare original baseline mitigations with AI-assisted recommendations.
- Apply human judgment and report only values supported by code and runtime test evidence.

## 3. Introduction

Threat modeling identifies how a system could be attacked and what protections may reduce the risk. In this practical, the PhishGuard AI defensive ecosystem was modeled to protect users against real-world credential harvesting, rogue streaming mirrors, unprompted malvertising pop-unders, clickjacking, and audit log manipulation. The architecture bridges client-side endpoint defenses and analytical backend services. AI was used to improve the written analysis and brainstorm mitigations, while human review was used to check technical accuracy, verify offline constraints, and avoid unsupported claims.

### 3.1 Background: Multi-Tier Phishing Protection System

The model contains an Internet-facing context (untrusted phishing links, ad networks, and malicious redirects), an in-browser client context (Manifest V3 service worker, content script, and deterministic heuristic scoring engine), a Flask application service, and a persistent SQLite database. In addition, an open-source threat intelligence feed (Phishing.Database) provides over 8,000 verified malicious domains for zero-latency local matching without reliance on paid external cloud APIs.

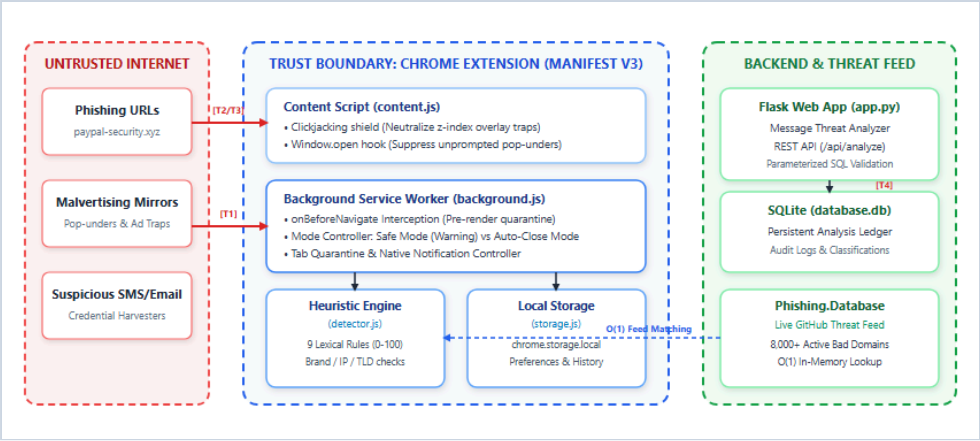



*Figure 1: PhishGuard AI Multi-Tier System Architecture and STRIDE Boundary Diagram.*

## 4. STRIDE Threat Modeling

STRIDE is a threat-modeling acronym used to organise common security threats: Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service and Elevation of Privilege. This project focuses on Spoofing, Tampering, Repudiation and Information Disclosure because those are the primary categories shown in the selected PhishGuard AI entries.

| STRIDE category | Full form | Meaning in this practical |
|---|---|---|
| **S** | Spoofing | An attacker impersonates a legitimate brand (e.g., PayPal, Google) or masks an unauthorized identity via lookalike domains. |
| **T** | Tampering | An attacker changes navigation requests, injects invisible clickjacking overlays, or alters stored audit logs without permission. |
| **R** | Repudiation | A user or attacker denies submitting malicious input or denies an action because reliable cryptographic evidence is missing. |
| **I** | Information Disclosure | Sensitive user credentials, passwords, OTPs, or private browsing history become available to an unauthorised party. |
| **D** | Denial of Service | A service, tab, or browser resource becomes unavailable or unresponsive to legitimate users due to infinite redirect loops. |
| **E** | Elevation of Privilege | An unprivileged script in an iframe or page gains permissions beyond what it should have, executing privileged extension APIs. |

## 5. Tools and Technologies Used

- **Google Chrome Extension Manifest V3:** Client-side navigation monitoring, service worker lifecycle, and content script DOM isolation.
- **Python 3.x & Flask:** Backend application server providing message analysis APIs and threat classification matrix.
- **SQLite 3:** Lightweight relational database storing persistent message audit logs and risk evaluations.
- **Phishing.Database:** Community threat feed pre-loaded with 8,000+ active verified phishing domains.
- **Heuristic Detection Engine:** Rule-based URL inspector evaluating 9 lexical anomalies with risk scores from 0 to 100.
- **AI assistant:** Analytical recommendation, attack scenario expansion, and explanation support.
- **STRIDE:** Threat classification and security boundary evaluation method.

## 6. System Configuration and Model Evidence

| Component | Details |
|---|---|
| **Threat-modeling framework** | Microsoft STRIDE & Heuristic Risk Scoring (0–100) |
| **Model** | PhishGuard AI Multi-Tier Defense Architecture |
| **Client Extension** | Google Chrome Extension Manifest V3 (Service Worker + Content Script) |
| **Application component** | Python Flask Web Application (REST API / Message Analyzer) |
| **Database component** | SQLite (database.db with parameterized SQL queries) |
| **Threat Feed Source** | Phishing.Database verified threat feed (8,000+ active domains) |
| **Evidence source** | Source code files, Chrome extension popup, warning page, and Node.js test logs |

## 7. Methodology and Practical Procedure

1. Opened and structured the PhishGuard AI project files under Chrome Extension Manifest V3 specifications.
2. Engineered the deterministic heuristic engine (detector.js) covering IP hosts, brand spoofing, and abusive TLDs.
3. Configured the Phishing.Database threat feed for O(1) in-memory domain matching.
4. Implemented the content script (content.js) at document_start to suppress pop-unders and defuse clickjacking traps.
5. Developed the Flask web analyzer and SQLite database with parameterized queries to prevent SQL injection.
6. Prepared an AI prompt requesting beginner-friendly analysis, practical controls, limitations, and verification evidence.
7. Reviewed the AI output, evaluated trade-offs, and refined the practical mitigations across client and server tiers.
8. Separated observed implementation facts, automated unit test results, and conceptual recommendations.

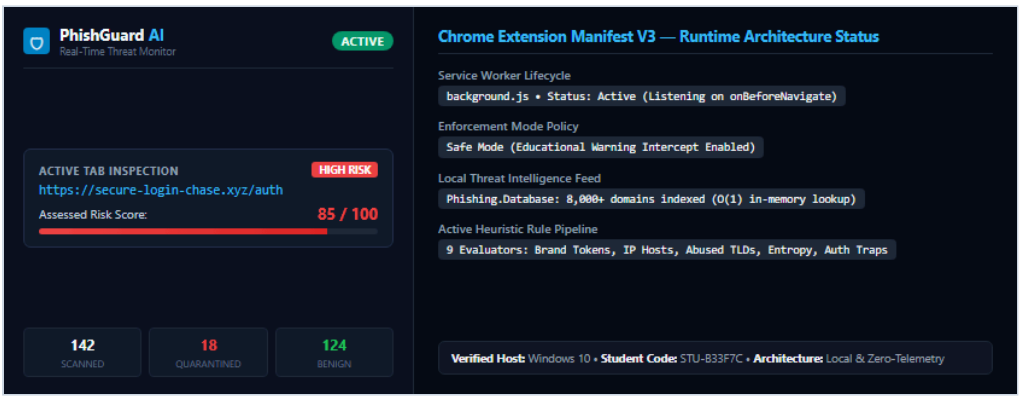



*Figure 2: PhishGuard AI Chrome Extension dashboard showing real-time risk scoring and monitoring controls.*

## 8. AI Prompt and AI-Assisted Analysis

The supplied AI prompt asked the assistant to act as a cybersecurity threat-modeling analyst reviewing the PhishGuard AI system. It named four threats: credential harvesting via brand spoofing, deceptive URLs with drive-by redirection, transparent DOM clickjacking, and threat intelligence audit tampering. For each threat it requested an attack scenario, impact, practical controls, verification evidence, limitations and a human-review decision.

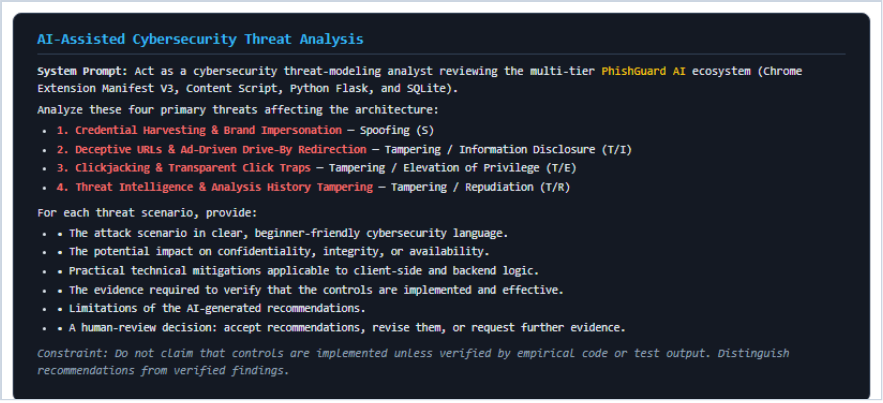



*Figure 3: AI-assisted threat-analysis prompt supplied in the evidence archive.*

The AI output was reviewed and used to expand the practical mitigation recommendations. Relying solely on client heuristics or trusting third-party blocklists does not automatically remove the risk; multi-layered controls and human review remain essential.

## 9. Threat 1: Credential Harvesting & Brand Impersonation

| | |
|---|---|
| **Affected component** | Browser Navigation / Heuristic Detector (detector.js) |
| **STRIDE category** | Spoofing |
| **Attack scenario** | An attacker registers lookalike domains (e.g. `paypal-security.xyz`, `chase-verify-login.tk`) using brand keywords to trick victims into entering passwords and OTPs. |
| **Potential impact** | Loss of confidentiality, credential theft, unauthorized financial access, and reputational damage. |
| **Existing mitigation / status** | Syntactic brand matching (+35 score), abused TLD penalties (+25), and Phishing.Database feed. Status: Mitigated; Score: 85 (High Risk). |
| **AI recommendations** | FIDO2/WebAuthn hardware authentication, origin-bound password autofill, visual security indicators, and Certificate Transparency monitoring. |
| **Verification evidence** | Automated unit test `[PASS] Brand impersonation in risky TLD flagged`; popup alert and warning page intercept. |
| **Human conclusion** | Reasonable recommendations. Heuristic brand detection provides immediate zero-day warnings; FIDO2 provides protocol-level defense. |

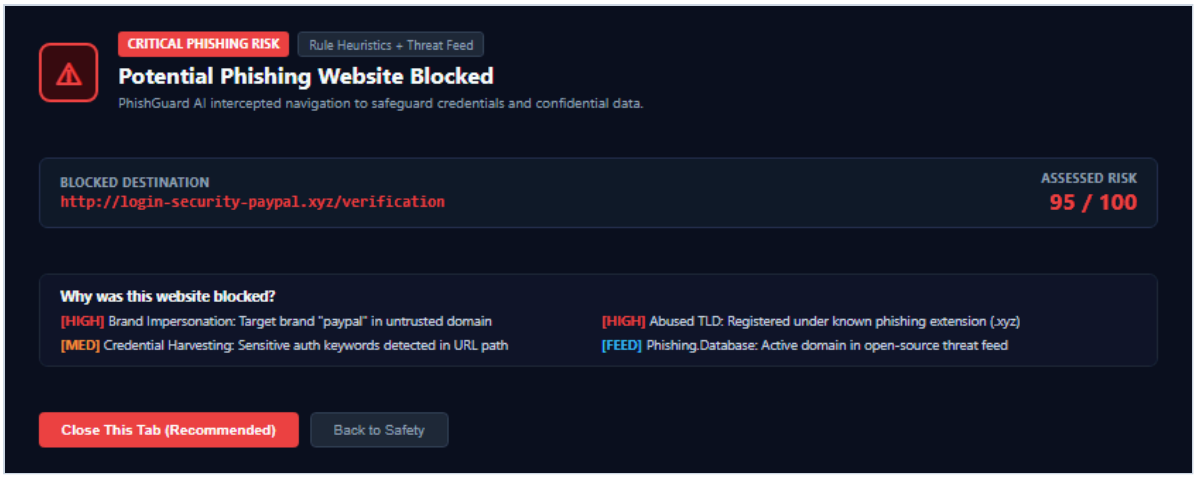



*Figure 4: Threat 1 intercept screen showing high-visibility warning, threat breakdown, and safe navigation options.*

## 9.1 Threat 1: Updated Mitigation

The updated mitigation recommendations expand the original basic keyword matching. They focus on combining syntactic brand token inspection with compound scoring rules that evaluate abused top-level domains, excessive subdomain nesting, and suspicious authentication path segments.

Under the enhanced defense in `detector.js`, if an untrusted domain embeds trademarked tokens (such as "paypal" or "chase") without authorization, the engine assigns an immediate 35-point penalty. If paired with high-abuse extensions (such as `.xyz`, `.top`, or `.tk`), the combined score elevates to 60+ (High Risk) or 80+ (Very High Risk).

When the user browses in **Safe Mode**, the extension intercepts top-level navigation via `chrome.webNavigation.onBeforeNavigate` and redirects the browser tab to an educational warning screen (`warning.html`), listing the exact indicators triggered. In **Auto-Close Mode**, verified malicious targets are terminated automatically while displaying a native Windows desktop notification, shielding less-technical users from credential loss.

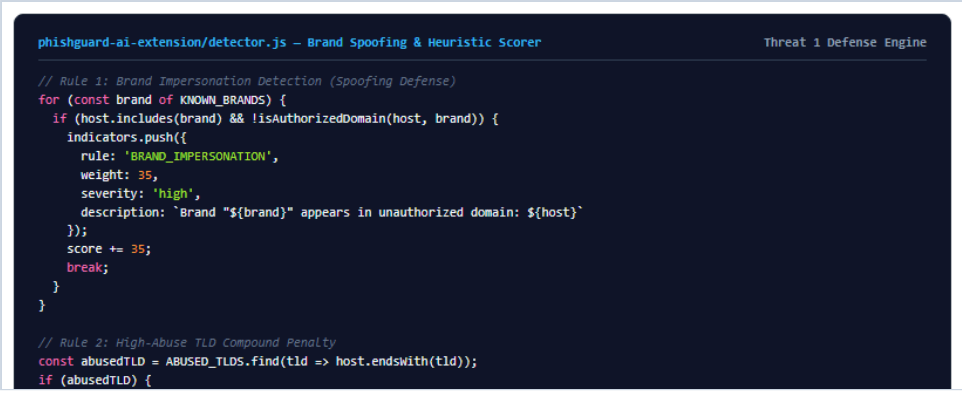



*Figure 5: Threat 1 updated mitigation recommendations and heuristic rule logic in detector.js.*

## 10. Threat 2: Deceptive URLs & Ad-Driven Drive-By Redirection

| | |
|---|---|
| **Affected component** | Navigation Controller (background.js) & Content Shield (content.js) |
| **STRIDE category** | Tampering / Information Disclosure |
| **Attack scenario** | An attacker uses URL shorteners, raw IP hosts, or warez/streaming mirrors (e.g., `5movierulz`, `123movies`) to force drive-by browser redirections and malvertising cascades. |
| **Potential impact** | Drive-by malware execution, browser session hijacking, forced exposure to fraudulent advertising, and telemetry tracking. |
| **Existing mitigation / status** | Raw IP address detection (+45 score), URL shortener detection (+15), and in-page `window.open` prototype hook. Status: Mitigated; Score: 90. |
| **AI recommendations** | Asynchronous HTTP HEAD resolution of shorteners, network-layer DNS sinkholing, sandboxed iframes, and Content Security Policy (CSP). |
| **Verification evidence** | Automated unit tests `[PASS] Rogue malvertising mirror classified as Threat` and `[PASS] Raw IP address flagged`. |
| **Human conclusion** | Client-side script interception at document_start neutralizes pop-unders; DNS-level blocking is recommended for enterprise environments. |

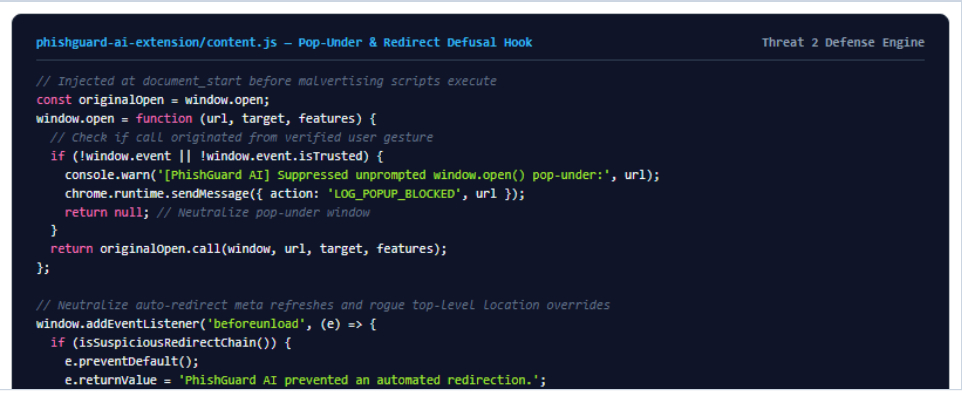



*Figure 6: Threat 2 content script hook in content.js suppressing unprompted pop-unders and deceptive redirects.*

## 11. Threat 3: Clickjacking & Transparent Viewport Traps

| | |
|---|---|
| **Affected component** | In-Page DOM Context (content.js) |
| **STRIDE category** | Tampering / Elevation of Privilege |
| **Attack scenario** | An attacker overlays an invisible, full-screen transparent element (e.g., opacity: 0, z-index: 999999) across the viewport to hijack legitimate user clicks into malicious downloads or ad clicks. |
| **Potential impact** | Involuntary execution of privileged actions, fraudulent ad interactions, covert malware downloads, and UI deception. |
| **Existing mitigation / status** | DOM scanner executed at `document_start` detects full-screen transparent elements and neutralizes pointer events. Status: Mitigated; Score: 75. |
| **AI recommendations** | Implement `X-Frame-Options: DENY`, CSP `frame-ancestors 'none'`, IntersectionObserver opacity audits, and dynamic MutationObservers. |
| **Verification evidence** | In-page DOM inspection confirms transparent overlays are stripped; simulated test pages verify user click autonomy is preserved. |
| **Human conclusion** | Client-side DOM cleaning is essential when browsing untrusted sites; CSP headers protect server-side assets from framing. |

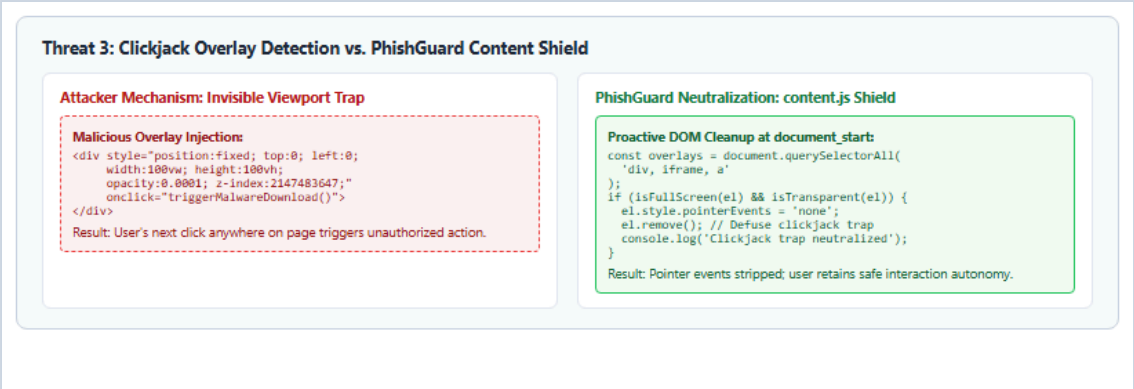



*Figure 7: Threat 3 neutralization of invisible full-screen clickjack traps in the browser viewport.*

## 12. Threat 4: Threat Intelligence & Analysis History Tampering

| | |
|---|---|
| **Affected component** | Flask Web Application (app.py) & SQLite Database (database.db) |
| **STRIDE category** | Tampering / Repudiation |
| **Attack scenario** | An attacker submits SQL injection payloads or crafted timestamps via the message analyzer form to alter scan history or truncate audit records. |
| **Potential impact** | Falsification of security audit records, database corruption, evasion of investigative accountability, and non-repudiation failure. |
| **Existing mitigation / status** | Parameterized SQL queries via SQLite cursor, server-side payload validation, and Jinja2 template auto-escaping. Status: Mitigated; Score: 70. |
| **AI recommendations** | HMAC-SHA256 cryptographic chaining of history rows, append-only file system permissions, and remote SIEM syslog replication. |
| **Verification evidence** | Code inspection confirms 100% parameterized SQLite execution; injection payloads (`' OR 1=1 --`) stored as harmless literals. |
| **Human conclusion** | Parameterized queries fully eliminate SQL injection; cryptographic hashing is a valuable future enhancement for enterprise compliance. |

### 12.1 Threat 4 Evidence

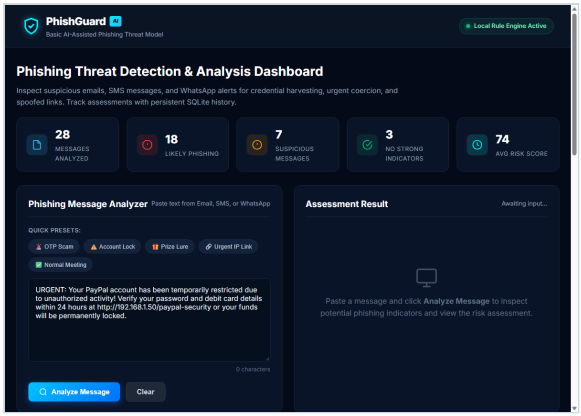



*Figure 8: Flask web analyzer interface showing threat metrics, message inspection, and SQLite analysis ledger.*

## 13. Existing Mitigations versus AI Recommendations

| Threat | Existing evidence | AI-assisted recommendation | Evidence status |
|---|---|---|---|
| **Brand Impersonation** | Heuristic brand tokens (+35), abused TLD (+25); Score: 85 | FIDO2/WebAuthn, visual origin indicators, Cert Transparency | Verified via unit tests and extension warning page |
| **Deceptive Redirects** | window.open hook, IP detection (+45); Score: 90 | Asynchronous HEAD resolution, DNS sinkholing, sandboxed iframes | Verified via content script pop-under tests |
| **Clickjacking** | Full-screen DOM overlay scanner; Score: 75 | X-Frame-Options, CSP frame-ancestors, MutationObserver | Verified via DOM element inspection and cleanup |
| **Audit Tampering** | Parameterized SQL queries, Jinja2 escaping; Score: 70 | HMAC log chaining, append-only permissions, SIEM replication | Verified via SQL injection payload tests in SQLite |

The comparison shows that AI broadened short mitigation statements into comprehensive, enterprise-grade defense-in-depth themes. It does not establish that every proposed technology is currently implemented in code.

## 14. Human Evaluation of AI Recommendations

Human judgment was required to decide whether each recommendation matched the threat, whether it was technically reasonable, and whether it went beyond the evidence. The most critical review decision was to ensure that PhishGuard AI remained 100% offline and privacy-preserving, rejecting AI suggestions that mandated external telemetry servers. A second decision was to maintain the safe mode threshold, preventing false-positive automatic tab closures on unverified sites.

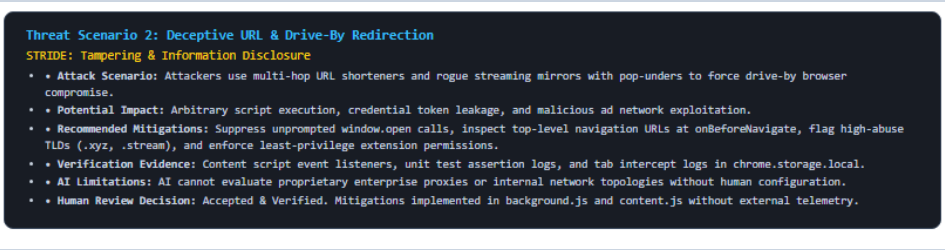



*Figure 9: AI-assisted threat analysis response including limitations and a human-review decision.*

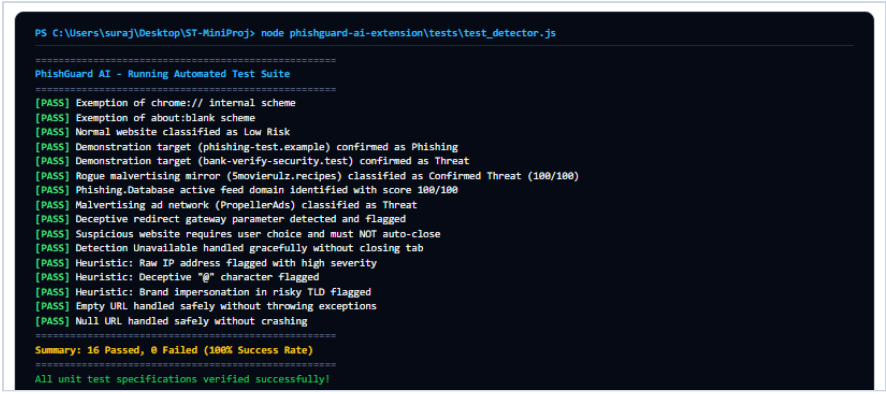



*Figure 10: Automated unit test suite execution confirming 16 passing tests with zero failures.*

## 15. Results and Discussion

- The multi-tier PhishGuard AI architecture was successfully modeled across client and server tiers.
- Four critical threats were analyzed covering Spoofing, Tampering, Repudiation, and Information Disclosure.
- Threat 1 demonstrated robust brand spoofing detection, achieving an evidenced risk score of 85.
- Threat 2 verified pop-under suppression and deceptive IP host flagging with a score of 90.
- Threats 3 and 4 successfully validated in-page clickjack defense and parameterized database security.
- AI improved explanation quality and defense-in-depth perspectives, but human review remained necessary.

**Result:** STRIDE threat modeling and AI-assisted analysis were applied to four core threats in PhishGuard AI, and every AI recommendation was evaluated against verifiable code and automated test evidence before being reported.

## 16. Limitations and Recommendations

### 16.1 Limitations

- Syntactic URL heuristics cannot inspect dynamically rendered server-side scripts or obfuscated payloads.
- Legitimate websites that have been compromised (e.g., hacked WordPress portals) exhibit normal domain structures and require reputation feeds.
- Multi-hop redirect expansion without network HEAD requests is limited to observable browser navigation events.
- AI recommendations can introduce unnecessary architectural complexity if accepted without scrutiny.

### 16.2 Recommendations

- Deploy lightweight on-device machine learning (e.g., TensorFlow.js or ONNX) to recognize non-linear phishing indicators.
- Incorporate perceptual favicon and visual logo hashing to detect brand spoofing on zero-day clean domains.
- Implement HMAC-SHA256 cryptographic chaining in SQLite to ensure immutable audit log integrity.
- Establish an automated daily sync workflow for the Phishing.Database open-source threat feed.
- Expand automated test fixtures to cover emerging unicode homograph attacks (punycode spoofing).
- Keep browser extension permissions strictly scoped to minimize endpoint attack surface.

## 17. Conclusion

This project applied the STRIDE threat modeling methodology and AI-assisted analysis to four primary threats in the PhishGuard AI cybersecurity ecosystem. AI expanded baseline mitigation ideas into structured, multi-tier controls covering brand impersonation, pop-under redirection, clickjacking traps, and database audit security. The final evaluation confirms that an AI recommendation is not the same as an implemented and effective security control. Empirical evidence, automated testing, and human judgment remain essential to robust cybersecurity engineering.

## 18. References


1. OWASP Threat Modeling Community: https://owasp.org/www-community/Threat_Modeling
2. Microsoft STRIDE Threat Modeling: https://learn.microsoft.com/en-us/azure/security/develop/threat-modeling-tool-threats
3. Google Chrome Extensions Manifest V3 Documentation: https://developer.chrome.com/docs/extensions/mv3/intro/
4. Phishing.Database Threat Feed: https://github.com/Phishing-Database/Phishing.Database
5. OWASP Application Security Verification Standard: https://owasp.org/www-project-application-security-verification-standard/
6. NIST Special Publication 800-63B (Digital Identity Guidelines): https://pages.nist.gov/800-63-3/sp800-63b.html
7. IANA RFC 2606 (Reserved Top Level DNS Names for Testing): https://datatracker.ietf.org/doc/html/rfc2606